Universidad de Guadalajara
Centro Universitario de Ciencias Económico-Administrativas
Maestría en Ciencias de los Datos | 2° Semestre
Materia: Almacenamiento de Grandes Bases de Datos

Instalacion de Paquetes para Grandes Bases de Datos en R: bigmemory y ff

Alumno:	Carlos Pulido Rosas
Fecha:	10 de mayo de 2026
Actividad:	Tarea 7 — Instalacion de paquetes bigmemory y ff en R
Herramienta:	R 4.5.3 (2026-03-11 ucrt)
Versión:	bigmemory 4.6.4 | ff 4.5.2
Entorno:	Windows 11 — ThinkPad T14 Gen 4homelab, Docker, etc.]
Dataset:	CRAN — Repositorio oficial de Rdataset]

1. Introduccion

El ecosistema de R ofrece herramientas especializadas para trabajar con conjuntos de datos que superan la capacidad de la memoria RAM disponible. Dos de los paquetes mas utilizados para este proposito son bigmemory y ff, cada uno con un enfoque diferente para gestionar grandes volumenes de datos de forma eficiente.

bigmemory implementa la estrategia de memoria compartida: crea matrices en una region de memoria que puede ser accedida simultaneamente por multiples procesos de R, lo que lo hace ideal para computo paralelo en un mismo equipo. ff, por su parte, utiliza archivos de mapa de memoria (memory-mapped files): los datos residen en disco pero se acceden como si estuvieran en RAM, gracias al mecanismo de paginacion del sistema operativo.

En esta actividad se documenta la instalacion de ambos paquetes en R 4.5.3 sobre Windows 11, la verificacion de su funcionamiento mediante ejemplos basicos, y una comparacion de las estrategias que cada uno implementa.

2. Instalacion de Paquetes

La instalacion se realiza desde el repositorio oficial CRAN mediante la funcion install.packages(). Ademas de bigmemory y ff, el gestor de dependencias instala automaticamente los paquetes requeridos por cada uno:

2.1 Comando de instalacion

```
install.packages(c("bigmemory", "ff"),
                 repos = "https://cran.rstudio.com/")
```

Instalacion de ambos paquetes desde CRAN en una sola instruccion.

Las dependencias instaladas automaticamente fueron:

Paquete	Version	Requerido por	Proposito
bigmemory.sri	0.1.8	bigmemory	Indice compartido para big.matrix

Rcpp	1.1.1	bigmemory	Interfaz C++ para R
uuid	1.2-2	bigmemory	Generacion de identificadores unicos
BH	1.90.0	bigmemory	Cabeceras Boost para C++
bit	4.6.0	ff	Vectores de bits de alta eficiencia

Tabla 1. Dependencias instaladas automaticamente junto con bigmemory y ff.

2.2 Verificacion de versiones instaladas

```
packageVersion("bigmemory")   # Verificar version
packageVersion("ff")
```

```
=== Verificacion de paquetes ===
bigmemory:  4.6.4
ff:         4.5.2
```

Salida real de R 4.5.3 — ambos paquetes instalados correctamente.

3. Paquete bigmemory — Memoria Compartida

bigmemory permite crear matrices de gran tamano almacenadas fuera del heap de R, en una region de memoria compartida del sistema operativo (shared memory). Multiples sesiones o procesos de R pueden acceder y modificar la misma matriz de forma concurrente, lo que lo hace especialmente util para procesamiento paralelo con los paquetes foreach o parallel.

3.1 Ejemplo: creacion y acceso a una big.matrix

```
library(bigmemory)

# Crear matriz de 1,000 filas x 5 columnas en memoria compartida
bm <- big.matrix(nrow = 1000, ncol = 5, type = "double", init = 0)

# Asignar valores a un subconjunto
bm[1:3, 1:3] <- matrix(1:9, 3, 3)

cat("Clase:", class(bm))
cat("Dimensiones:", nrow(bm), "x", ncol(bm))
print(bm[1:3, 1:3])

# Obtener el descriptor para compartir con otros procesos
desc <- describe(bm)
```

```
Clase: big.matrix
Dimensiones: 1000 x 5
Primeras 3 filas:
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

big.matrix de 1,000 x 5 en memoria compartida. Los datos persisten fuera del heap de R.

La funcion describe() genera un objeto que puede pasarse a otros procesos de R para que accedan a la misma region de memoria mediante attach.big.matrix(), sin necesidad de copiar los datos.

4. Paquete ff — Archivos de Mapa de Memoria

ff implementa la estrategia de memory-mapped files: los datos se almacenan en un archivo binario en disco y el sistema operativo los mapea al espacio de direcciones del proceso. Desde el punto de vista del programador, el acceso es identico al

de un vector o arreglo en RAM, pero el consumo real de memoria es minimo porque el SO carga en RAM unicamente las paginas que se estan usando en cada momento.

4.1 Ejemplo: creacion y acceso a un vector ff

```
library(ff)

# Crear un vector ff de 10,000 enteros mapeado a disco
tmp <- tempfile(fileext = ".ff")
ffv <- ff(1:10000, filename = tmp, vmode = "integer")

cat("Clase:", class(ffv))
cat("Longitud:", length(ffv))
cat("Primeros 5 elementos:", ffv[1:5])
cat("Suma de todos los elementos:", sum(ffv[]))
cat("Tamano en disco (bytes):", file.info(filename(ffv))$size)

close(ffv)   # Liberar el mapa de memoria
```

```
=== ff: Archivos de mapa de memoria ===
Clase: ff_vector ff
Longitud: 10000
Primeros 5 elementos: 1 2 3 4 5
Suma de todos los elementos: 50005000
Tamano en disco (bytes): 40000
```

Vector ff de 10,000 enteros. Los 40,000 bytes corresponden a 4 bytes por entero (vmode='integer').

El parametro vmode controla el tipo de dato a nivel binario. Con vmode='integer' cada elemento ocupa exactamente 4 bytes, de ahi los 40,000 bytes en disco para 10,000 elementos. ff soporta tipos enteros de 1, 2, 4 y 8 bytes, reales de 4 y 8 bytes, y valores logicos y categoricos.

5. Comparacion de Estrategias

Caracteristica	bigmemory	ff
Estrategia	Memoria compartida (shared memory)	Archivos de mapa de memoria (mmap)
Ubicacion de datos	RAM / shared memory del SO	Archivo binario en disco
Acceso concurrente	Si — multiples procesos R	Limitado (un proceso por defecto)
Persistencia	Opcional (backing file)	Si — el archivo persiste en disco
Estructura soportada	Matrices (big.matrix)	Vectores, matrices, data frames (ffdf)
Tipo de datos	numeric, integer, short, char	integer, double, logical, factor y mas
Uso tipico	Computo paralelo en un equipo	Datasets que no caben en RAM
Version instalada	4.6.4	4.5.2

Tabla 2. Comparacion de bigmemory y ff segun sus caracteristicas principales.

6. Conclusiones

Ambos paquetes fueron instalados y verificados correctamente en R 4.5.3 sobre Windows 11. Los ejemplos de uso confirman que las estructuras de datos creadas operan como se espera: big.matrix almacena y recupera valores desde memoria compartida, y el vector ff mapea correctamente 10,000 enteros a un archivo de 40,000 bytes en disco.

- **Complementariedad.** bigmemory y ff no son competidores sino complementarios: bigmemory es la eleccion natural cuando se necesita paralelismo en un mismo equipo, mientras que ff es preferible cuando el dataset supera la RAM disponible y no se requiere acceso concurrente.
- **Transparencia de acceso.** Ambos paquetes exponen una interfaz de subindice identica a la de los objetos nativos de R (vectores y matrices), lo que minimiza la curva de aprendizaje.

- **Relevancia para grandes bases de datos.** En el contexto de la materia, estas estrategias son el equivalente en R a lo que HDFS y el particionado representan en Hive: mecanismos para procesar datos que rebasan la capacidad de la memoria principal del equipo local.
- **Preparacion para la siguiente sesion.** Con bigmemory y ff instalados y verificados, el entorno local esta listo para aplicar tecnicas de analisis estadistico sobre datasets de gran escala directamente en R, sin requerir infraestructura distribuida adicional.